

This is basically full-stack development.

Goal:

We can test 2 approaches:

1. HTML & CSS (Frontend), Node.js (Backend)
2. React.js (Frontend) & Node.js (Backend) – (Abandon)

We test the first approach first, then we can swap the approach 1 frontend with the approach 2 frontend ([React.js](#)).

Review System Info

Nginx acts as a Reverse Proxy in the Public Subnet to handle incoming traffic, while **Apache2** runs the Backend API in the Private Subnet. The backend communicates with RDS MySQL and uses a NAT Gateway for outbound updates.

Proxy Types

In my setup, I actually have two different types of proxies working for different reasons:

- The Reverse Proxy (Nginx/Apache): * Likely sitting on your Backend EC2 instance.
 - Purpose: It "fronts" your Node.js app. When a user hits your IP on port 80 (HTTP), Nginx receives the request and "proxies" it to your backend app running on a different port (like 3000 or 5000).
 - Why? It handles security, SSL, and buffering better than Node.js does directly.
- The NAT Gateway (Forward Proxy Behavior): * Sitting in your Public Subnet.
 - Purpose: It allows your Private Instance (like your Database or a private Backend) to go out to the internet to download updates or talk to Elastic Fleet without letting the internet come in.

Programming Language

In my frontend, I used HTML, CSS, and JS.

In my backend, I used Node.js and Express.js.

We used Express, AWS SDK, and MySQL2. Reasons can be found at the bottom of this documentation called Reason Approach #1.

Approach #1

HTML & CSS (Frontend), Node.js (Backend)

Frontend (Public Instance / Reverse Proxy)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Company Cloud Drive</title>
```

```
<style>
```

```
body { font-family: sans-serif; background: #f0f2f5; padding: 50px; }
```

```
.box { background: white; padding: 20px; border-radius: 10px; box-shadow: 0 4px 10px  
rgba(0,0,0,0.1); }
```

```
input, button { padding: 10px; margin: 5px 0; }
```

```
table { width: 100%; border-collapse: collapse; margin-top: 20px; }
```

```
th, td { border: 1px solid #ddd; padding: 10px; text-align: left; }
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="box">
```

```
<h2>🚀 Company File Manager</h2>
```

```
<input type="file" id="fileInput">
```

```
<button onclick="uploadFile()">Upload</button>
```

```
<hr>
```

```
<input type="text" id="searchBox" placeholder="Search files..." oninput="loadFiles()">
```

```
<table id="fileTable">
```

```
<thead><tr><th>File</th><th>Link</th><th>Action</th></tr></thead>
```

```
<tbody></tbody>
```

```
</table>
```

```
</div>
```

```
<script>
```

```
const API_URL = "http://[PRIVATE-INSTANCE-INTERNAL-IP]:3000/api";
```

```
async function loadFiles() {
```

```
  const search = document.getElementById('searchBox').value;
```

```
  const res = await fetch(`${API_URL}/files?search=${search}`);
```

```
  const files = await res.json();
```

```
  const tbody = document.querySelector('#fileTable tbody');
```

```
  tbody.innerHTML = files.map(f => `
```

```
    <tr>
```

```
      <td>${f.filename}</td>
```

```
      <td><a href="${f.s3_url}" target="_blank">View</a></td>
```

```
      <td><button onclick="deleteFile(${f.id})"
style="color:red">Delete</button></td>
```

```
    </tr>
```

```
  `).join("");
```

```
}
```

```
async function uploadFile() {  
  
    const file = document.getElementById('fileInput').files[0];  
  
    const formData = new FormData();  
  
    formData.append('file', file);  
  
    await fetch(`${API_URL}/upload`, { method: 'POST', body: formData });  
  
    loadFiles();  
  
}
```

```
async function deleteFile(id) {  
  
    await fetch(`${API_URL}/delete/${id}`, { method: 'DELETE' });  
  
    loadFiles();  
  
}
```

```
loadFiles(); // Initial load
```

```
</script>
```

```
</body>
```

```
</html>
```

Backend (Private Instance/Server)

```
mkdir company-api && cd company-api
```

```
sudo apt install npm -y
```

```
npm init -y
```

```
npm install express mysql2 @aws-sdk/client-s3 multer
```

Then,

```
node -v (check version)
```

```
ls
```

You should see these files:

```
ubuntu@ip-10-0-2-37:~/company-api$ ls
node_modules package-lock.json package.json
ubuntu@ip-10-0-2-37:~/company-api$ node -v
v18.19.1
ubuntu@ip-10-0-2-37:~/company-api$
```

Next, create

```
vi server.js
```

==(Click i to start editing)==>

```
const express = require('express');
```

```
const mysql = require('mysql2');
```

```
const { S3Client, PutObjectCommand, DeleteObjectCommand } =
require('@aws-sdk/client-s3');
```

```
const multer = require('multer');
```

```
const app = express();
```

```
const upload = multer({ storage: multer.memoryStorage() });
```

```
// =====
```

```
// CONFIG
```

```
// =====
```

```
const BUCKET = "company-storage-network-project";
```

```

const s3 = new S3Client({

  region: "{INSERT-YOUR-REGION}"

  // Using IAM Role on the EC2 instance – do NOT hardcode keys.Go to
  Amazon S3 > General Purpose Buckets > Your-bucket > AWS Region

});

const db = mysql.createConnection({

  host: '{INSERT-DB-ENDPOINT}', // RDS Service > Databases > Click your DB
  > Connectivity & Security tab (e.g.,
  company-db.xyz.us-east-1.rds.amazonaws.com)

  user: 'admin',

  password: '{INSERT-DB-PASSWORD}',

  database: '{INSERT-DB-NAME}' // You can find this after login via
  private instance CLI and type: "show databases;" It is located at the first
  row.

});

db.connect(err => {

  if (err) {

    console.error("DB connection failed:", err);

    process.exit(1);

  }

  console.log("✅ Database connected");

});

app.use(express.json());

// Logger

app.use((req, res, next) => {

  console.log(`${new Date().toISOString()} - ${req.method} ${req.url}`);

```

```

    next();
  });

// =====

// ROUTES

// =====

// GET FILES (Search)

app.get('/api/files', (req, res) => {

  const search = req.query.search || '';

  const sql = `

    SELECT id, filename, s3_url, upload_date

    FROM uploads

    WHERE filename LIKE ?

    ORDER BY upload_date DESC

  `;

  db.query(sql, [`%${search}%`], (err, results) => {

    if (err) return res.status(500).json({ error: err.message });

    res.json(results);

  });

});

// UPLOAD FILE

app.post('/api/upload', upload.single('file'), async (req, res) => {

  if (!req.file) return res.status(400).json({ error: "No file uploaded"
});

```

```
    const file = req.file;

    // Make S3 key unique to avoid overwriting
    const key = `${Date.now()}-${file.originalname}`;

    try {

        // Upload to S3

        await s3.send(new PutObjectCommand({

            Bucket: BUCKET,

            Key: key,

            Body: file.buffer,

            ContentType: file.mimetype

        })));

        const s3Url = `https://${BUCKET}.s3.amazonaws.com/${key}`;

        // Save metadata in DB (only columns we have)

        const sql = "INSERT INTO uploads (filename, s3_url) VALUES (?, ?)";

        db.query(sql, [file.originalname, s3Url], (err) => {

            if (err) return res.status(500).json({ error: err.message });

            res.json({ message: "Upload successful", file: { filename:
file.originalname, s3_url: s3Url } });

        });

    } catch (err) {

        console.error(err);

        res.status(500).json({ error: err.message });

    }

});
```



```
// DELETE FILE

app.delete('/api/files/:id', (req, res) => {

  const id = req.params.id;

  db.query("SELECT s3_url FROM uploads WHERE id = ?", [id], async (err,
results) => {

    if (err) return res.status(500).json({ error: err.message });

    if (results.length === 0)

      return res.status(404).json({ error: "File not found" });

    const s3Url = results[0].s3_url;

    const key = s3Url.split('.com/')[1];

    try {

      // Delete from S3

      await s3.send(new DeleteObjectCommand({

        Bucket: BUCKET,

        Key: key

      }));

      // Delete from DB

      db.query("DELETE FROM uploads WHERE id = ?", [id], (err) => {

        if (err) return res.status(500).json({ error: err.message
});

        res.json({ message: "Deleted successfully" });

      });

    } catch (err) {
```

```

        res.status(500).json({ error: err.message });
    }

    });
});

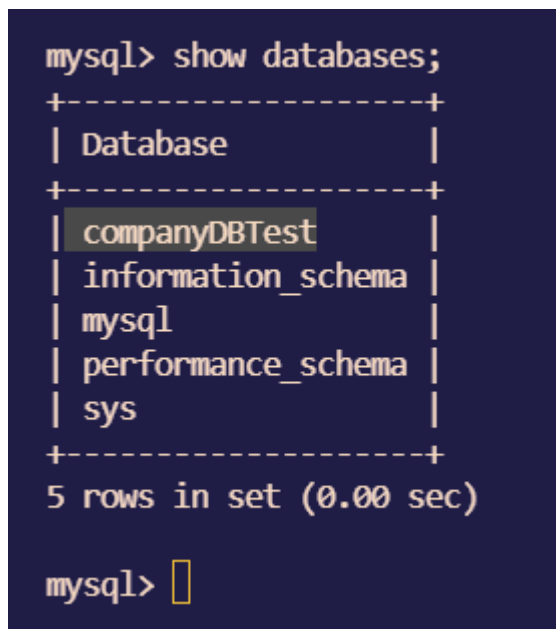
// =====

app.listen(3000, '0.0.0.0', () => {
    console.log("✅ API running on port 3000");
});

```

Note:

For the database name, this is where you should get it from:



```

mysql> show databases;
+-----+
| Database                |
+-----+
| companyDBTest           |
| information_schema       |
| mysql                   |
| performance_schema      |
| sys                     |
+-----+
5 rows in set (0.00 sec)

mysql> 

```

It is the first one appear when you first enter mysql console. After editing server.js, exit by typing (Esc > :x)

After copy-pasting codes, run this on ur Private Instance:

```
node server.js
```

You should see this:

```
ubuntu@ip-10-0-2-37:~/company-api$ vi server.js
ubuntu@ip-10-0-2-37:~/company-api$ node server.js
(node:13145) Warning: NodeDeprecationWarning: The AWS SDK for JavaScript (v3) will
no longer support Node.js v18.19.1 in January 2026.

To continue receiving updates to AWS services, bug fixes, and security
updates please upgrade to a supported Node.js LTS version.

More information can be found at: https://a.co/c895JFp
(Use `node --trace-warnings ...` to show where the warning was created)
API ready on port 3000
█
```

Back on the Public Instance:

We need to configure the nginx system to talk to the private instance web server.

```
sudo nano /etc/nginx/sites-available/default
```

```
===>
```

```
server_name _;
```

```
# 1. THE FRONTEND (Serve index.html from this Public Instance)
```

```
location / {
```

```
    root /var/www/html;
```

```
    index index.html;
```

```
    try_files $uri $uri/ =404;
```

```
}
```

```
# 2. THE BACKEND (Send /api calls to the Private Instance)
```

```
location /api/ {
```

```
    proxy_pass http://10.0.2.37:3000/api/;
```

```
    proxy_set_header Host $host;
```

```
    proxy_set_header X-Real-IP $remote_addr;
```

```
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
}
```

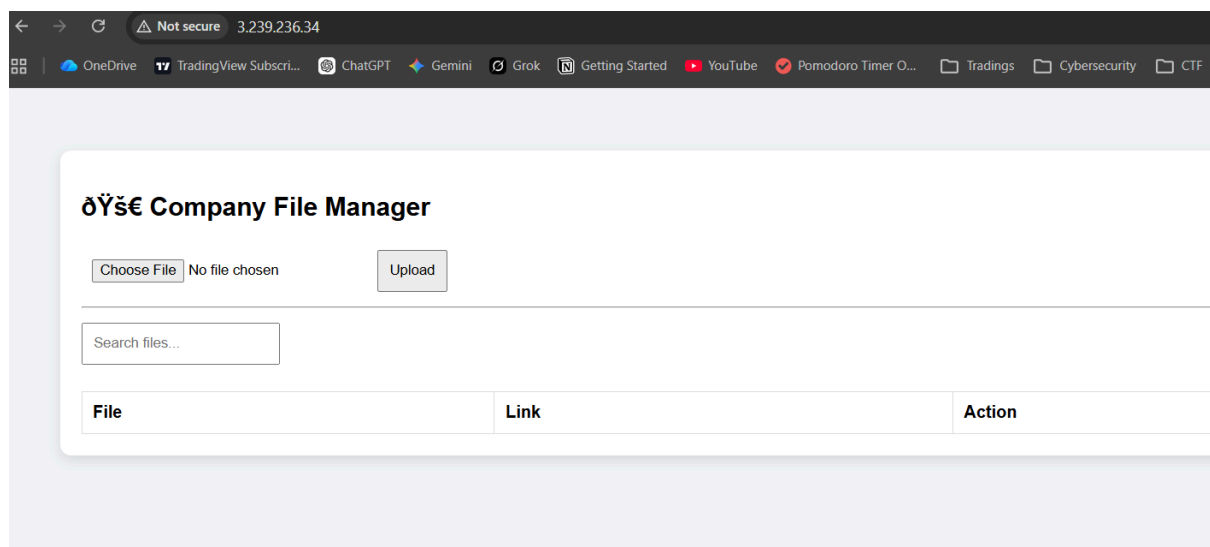
Save & exit. Then:

```
sudo chown www-data:www-data /var/www/html/index.html
```

```
sudo chmod 644 /var/www/html/index.html
```

Then, open the public instance IP.

Result:



Reasons Approach #1:

Why are we using express, aws sdk, and mysql2 here?

When you're building a professional application, you don't want to "reinvent the wheel." These three libraries are the industry-standard "tools" that allow your Node.js code to talk to the outside world.

Think of **Node.js** as the engine of a car. It provides the power, but it doesn't have a steering wheel, seats, or a GPS. These libraries provide those parts:

1. Express (The "Steering Wheel" & "Dashboard")

- **Why?** Pure Node.js is very difficult to use for web servers. If you used pure Node, you would have to manually write code to figure out if a user is visiting `/upload` or `/delete`, and manually parse every chunk of data they send.
- **What it does:** It provides **Routing**. It allows you to simply say `app.get('/search', ...)` or `app.post('/upload', ...)`. It makes your code readable and organized.

+1

2. AWS SDK (The "Universal Key")

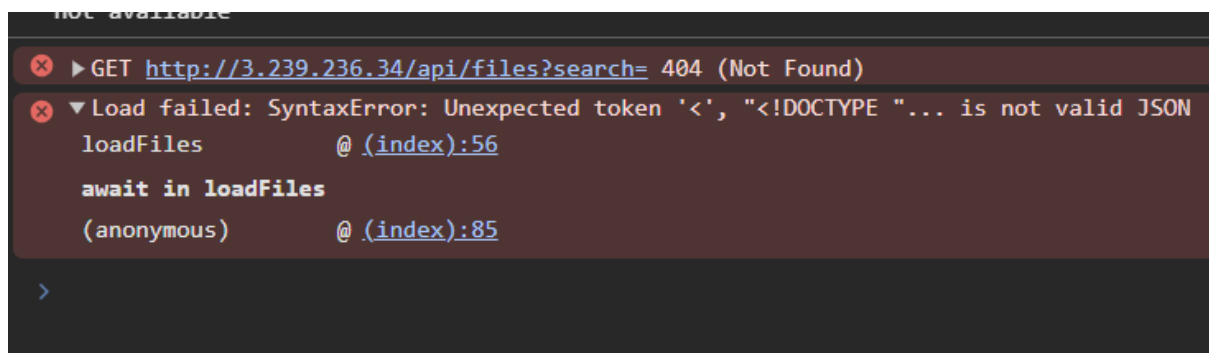
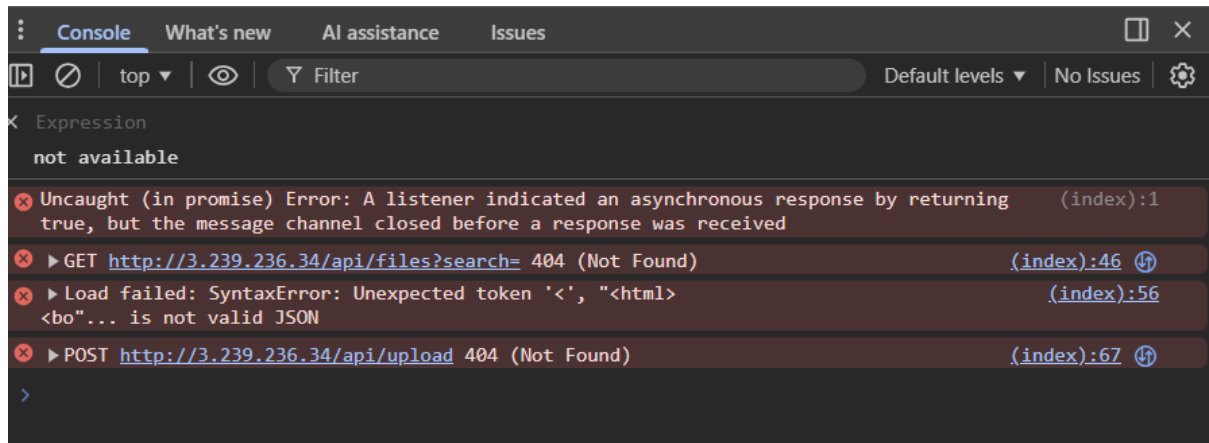
- **Why?** S3 is a secure, complex storage system. You can't just "send" a file to it like a regular folder. It requires specific encryption, headers, and authentication protocols (SigV4).
- **What it does:** It's the official library built by Amazon. It handles the **Authentication** (using your IAM Role automatically) and the **Communication** logic. Instead of writing 100 lines of code to send a file, you just use `s3.send(new PutObjectCommand(...))`.

3. MySQL2 (The "Translator")

- **Why?** Databases speak SQL, and Node.js speaks JavaScript. You need a driver to bridge that gap.
- **Why mysql2 instead of the older mysql?**
 - **Promises:** It supports `async/await`, which makes your code look clean and modern (instead of "callback hell").
 - **Security:** It has built-in support for **Prepared Statements**. This is critical because it prevents "SQL Injection" (where a hacker types a command into your search bar to delete your whole database).
 - **Performance:** It is faster and uses less memory than the original version.

Issue – Nginx Configuration

There was a time when these issues appeared. It was miserable. It took me about 3 days + 1 day of building the MySQL service to use it locally, mimicking this project, which is on the cloud. The issue has to do with the nginx configuration.



Solve:

The issue was with the nginx configuration. In the public instance, check the nginx configuration file. Go to this CLI: `sudo nano /etc/nginx/sites-available/default`

This is how the nginx config file should look:

```
# Default server configuration
#
server {
    listen 80 default_server;
    listen [::]:80 default_server;
```

```
root /var/www/html;
```

```
index index.html index.htm index.nginx-debian.html;
```

```
server_name _;
```

```
# Serve the HTML
```

```
location / {
```

```
    #root /var/www/html;
```

```
    #index index.html;
```

```
    try_files $uri /index.html;
```

```
}
```

```
# 1. THE REVERSE PROXY LOGIC | Proxy the API calls
```

```
location /api/ {
```

```
# Replace 10.0.2.XXX with your Private Instance's ACTUAL Private
```

```
    proxy_pass http://10.0.2.37:3000;
```

```
# Standard Proxy Headers
```

```
    proxy_set_header Host $host;
```

```
    proxy_set_header X-Real-IP $remote_addr;
```

```
}
```

- In the public instance
 - `curl http://10.0.2.37:3000/api/files`
 - If it shows “[]”, it means:

- ✓ Backend is working
- ✓ Private EC2 reachable
- ✓ Node is running
- ✓ Security group is already allowing traffic

- So your infrastructure is fine.
- Your problem is 100% NGINX config.

Make sure that these two are correctly communicating and programmed.

- server.js
- index.html

Make sure the Private Instance inbound rule is set to:

- Custom TCP
- Range port: 3000
- Source: Public Instance EC2 SG (Reverse Proxy)

The screenshot shows the AWS Management Console interface for editing inbound rules on a security group. The breadcrumb navigation at the top indicates the path: EC2 > Security Groups > sg-0e6ee758106ebfc38 - Private-Server > Edit inbound rules. The main heading is 'Edit inbound rules' with an 'Info' link. Below the heading is a note: 'Inbound rules control the incoming traffic that's allowed to reach the instance.'

The 'Inbound rules' section contains a table with the following columns: Security group rule ID, Type, Protocol, Port range, Source, and Description - optional. There are three rules listed:

Security group rule ID	Type	Protocol	Port range	Source	Description - optional
sgr-0cdfea142de9bb1ab	SSH	TCP	22	Custom	This is for updating the private s...
sgr-01ad5aba9d9523025	HTTP	TCP	80	Custom	This is for updating the private server or troubleshooting the private server.
sgr-06ca24349c60b278c	Custom TCP	TCP	3000	Custom	Communication with backend

Below the table is an 'Add rule' button. The 'Custom TCP' rule is selected, and a dropdown menu is open showing a list of source security groups. The dropdown list includes:

- default | sg-015c62fb44416b7b7
- Load-Balancer | sg-0e3874035fe5a7ce8
- Load-Balancer
- rds-ec2-1 | sg-0ec4...
- Company-DB (old/unused)
- Private-Server | sg-0e6ee758106ebfc38
- Private-Server
- ec2-rfsc-2 | sg-0r09f26904c2d6f9h

The 'sg-0e3874035fe5a7ce8' option is highlighted in the dropdown. At the bottom right of the console, there are 'Cancel' and 'Save rules' buttons.

Without this security group, no logs or error logs will appear in S3, server.js, and the browser developer console. This is the fixed result.

Search Files

Search

• Screenshot (416).png

Delete

Check with S3:

Before

Objects (2)

Copy S3 URI

Copy URL

Download

Open ↗

Delete

Actions ▾

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	testRole.txt	txt	February 3, 2026, 21:28:47 (UTC+07:00)	169.0 B	Standard
☐	Screenshot 2026-01-26 202913.png	png	January 28, 2026, 21:21:02 (UTC+07:00)	54.3 KB	Standard

After:

Objects (3)

Copy S3 URI

Copy URL

Download

Open ↗

Delete

Actions ▾

Create folder

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them perm [more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	1771044324715-Screenshot (416).png	png	February 14, 2026, 11:45:25 (UTC+07:00)	515.0 KB	Standard
☐	testRole.txt	txt	February 3, 2026, 21:28:47 (UTC+07:00)	169.0 B	Standard
☐	Screenshot 2026-01-26 202913.png	png	January 28, 2026, 21:21:02 (UTC+07:00)	54.3 KB	Standard

Check with Database:

```
Database changed
mysql> select * from uploads;
+----+-----+-----+-----+
| id | filename                               | s3_url                                                                 | upload_date |
+----+-----+-----+-----+
| 6  | Screenshot (416).png                  | https://company-storage-network-project.s3.amazonaws.com/1771044324715-Screenshot (416).png | 2026-02-14 04:45:24 |
+----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Suggestion #1: Run server.js onboot

Make `server.js` run on boot so you don't have to type it repeatedly. **This is for a simple server.**

```
cd /home/ubuntu
```

```
mkdir company-api/
```

```
cd company-api/
```

```
sudo nano server.js
```

Copy-paste JS code:

```
{Your-javascript-codes}
```

Save & Exist:

```
chmod +x server.js
```

Continue with:

```
sudo nano /etc/systemd/system/myscript.service
```

Copy-paste this code:

```
[Unit]
```

```
Description=Company API Server
```

```
After=network.target
```

```
[Service]
```

```
# Ensure /usr/bin/node is the correct path (check with 'which node')
```

```
ExecStart=/usr/bin/node /home/ubuntu/company-api/server.js
```

```
WorkingDirectory=/home/ubuntu/company-api/
```

```
Restart=always
```

```
# Crucial: waits 5s before restarting to prevent the "start-limit-hit" error you saw earlier
```

```
RestartSec=5
```

```
User=ubuntu
```

Environment=NODE_ENV=production

[Install]

WantedBy=multi-user.target

```
GNU nano 7.2 /etc/systemd/system/myscript.service *
[Unit]
Description=Company API Server
After=network.target

[Service]
# Ensure /usr/bin/node is the correct path (check with 'which node')
ExecStart=/usr/bin/node /home/ubuntu/company-api/server.js
WorkingDirectory=/home/ubuntu/company-api
Restart=always
# Crucial: waits 5s before restarting to prevent the "start-limit-hit" error you saw earlier
RestartSec=5
User=ubuntu
Environment=NODE_ENV=production
[]
[Install]
WantedBy=multi-user.target
```

Save & Exist, then:

sudo systemctl daemon-reload

sudo systemctl start myscript.service

sudo systemctl status myscript.service

```
ubuntu@ip-10-0-2-37:/opt/server$ sudo nano /etc/systemd/system/myscript.service
ubuntu@ip-10-0-2-37:/opt/server$ sudo nano /etc/systemd/system/myscript.service
ubuntu@ip-10-0-2-37:/opt/server$ sudo systemctl daemon-reload
ubuntu@ip-10-0-2-37:/opt/server$ sudo systemctl start myscript.service
ubuntu@ip-10-0-2-37:/opt/server$ sudo systemctl status myscript.service
● myscript.service - Company API Server
   Loaded: loaded (/etc/systemd/system/myscript.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-02-16 05:53:18 UTC; 4s ago
     Main PID: 8169 (node)
       Tasks: 14 (limit: 1017)
      Memory: 39.2M (peak: 39.5M)
         CPU: 593ms
    CGroup: /system.slice/myscript.service
            └─8169 /usr/bin/node /home/ubuntu/company-api/server.js

Feb 16 05:53:18 ip-10-0-2-37 systemd[1]: myscript.service: Scheduled restart job, restart counter is at 69.
Feb 16 05:53:18 ip-10-0-2-37 systemd[1]: Started myscript.service - Company API Server.
Feb 16 05:53:19 ip-10-0-2-37 node[8169]: (node:8169) Warning: NodeDeprecationWarning: The AWS SDK for JavaScript (v3) will
Feb 16 05:53:19 ip-10-0-2-37 node[8169]: no longer support Node.js v18.19.1 in January 2026.
Feb 16 05:53:19 ip-10-0-2-37 node[8169]: To continue receiving updates to AWS services, bug fixes, and security
Feb 16 05:53:19 ip-10-0-2-37 node[8169]: updates please upgrade to a supported Node.js LTS version.
Feb 16 05:53:19 ip-10-0-2-37 node[8169]: More information can be found at: https://a.co/c895JFp
Feb 16 05:53:19 ip-10-0-2-37 node[8169]: (Use `node --trace-warnings ...` to show where the warning was created)
Feb 16 05:53:19 ip-10-0-2-37 node[8169]: ✓ API running on port 3000
Feb 16 05:53:19 ip-10-0-2-37 node[8169]: ✓ Database connected
```

In real production, you should save the script files and their modules in the /opt directory. This directory is for hosting servers. Make sure you create another directory to store the server files. In this case, we created a server/ directory.

***Note:** You may need to do your own research, as this might flag errors.*

You can create a new server.js or move it into the /opt directory.

Moving Method:

```
sudo mv ~/company-api/package.json /opt/server/
```

```
sudo mv ~/company-api/package-lock.json /opt/server/
```

```
cd /opt/server
```

```
sudo chown -R ubuntu:ubuntu /opt/server
```

```
npm install
```

Continue with:

```
sudo nano /etc/systemd/system/myscript.service
```

Copy-paste this code:

```
[Unit]
```

```
Description=Company API Server
```

```
After=network.target
```

```
[Service]
```

```
# Ensure /usr/bin/node is the correct path (check with 'which node')
```

```
ExecStart=/usr/bin/node /opt/server/server.js
```

```
WorkingDirectory=/opt/server/
```

```
Restart=always
```

```
# Crucial: waits 5s before restarting to prevent the "start-limit-hit" error you saw earlier
```

```
RestartSec=5
```

```
User=ubuntu
```

```
Environment=NODE_ENV=production
```

[Install]

```
WantedBy=multi-user.target
```

```
GNU nano 7.2 /etc/systemd/system/myscript.service
[Unit]
Description=Company API Server
After=network.target

[Service]
# Ensure /usr/bin/node is the correct path (check with 'which node')
ExecStart=/usr/bin/node /opt/server/server.js
WorkingDirectory=/opt/server/
Restart=always
# Crucial: waits 5s before restarting to prevent the "start-limit-hit" error you saw earlier
RestartSec=5
User=ubuntu
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

Then,

```
sudo systemctl daemon-reload
```

```
sudo systemctl start myscript.service
```

```
sudo systemctl status myscript.service
```

Troubleshooting or watching logs:

```
journalctl -u myscript.service -f
```

Now, the server.js will run forever. Real web server. You can test this by closing Tabby or PuTTY. Then, refresh your web page and try uploading or interacting with the features. It should work just fine.

=====

END